Algorithmes et Pensée Computationnelle

Programmation orientée objet : Héritage et Polymorphisme

Le but de cette séance est d'approfondir les notions de programmation orientée objet vues précédemment. Nous commencerons par un rappel des notions de surcharge d'opérateurs avant d'introduire des exercices sur l'héritage et le polymorphisme. Au terme de cette séance, vous devez être en mesure de factoriser votre code afin de le rendre mieux structuré et plus lisible. Le langage utilisé est indiqué pour chaque exercice.

Le code présenté dans les énoncés se trouve sur Moodle, dans le dossier `Code`.

Les durées indiquées (🕓) sont données à titre indicatif.

1 Rappel : Surcharge des opérateurs en Python

Dans cette section, vous manipulerez des fractions sous forme d'objets. Vous ferez des opérations de base sur ce nouveau type d'objets.

Question 1: (🕓 *5 minutes*) **Création de classe**
Dans un projet que vous aurez au préalable préparé, créez un fichier `surcharge.py`. Définissez-y une classe `Fraction` possédant deux attributs privés : le numérateur et le dénominateur.

Question 2: (🕓 *5 minutes*) **Constructeur et valeurs par défaut**
Définissez un constructeur acceptant un numérateur et un dénominateur, dont les valeurs par défaut sont respectivement 0 et 1. Avec un seul argument, `Fraction(n)` doit représenter la fraction $\frac{n}{1}$. Levez une exception `ZeroDivisionError` si le dénominateur est 0.

Question 3: (🕓 *5 minutes*) **Conversion en entiers**
Convertissez les valeurs des attributs en entiers.

Question 4: (🕓 *5 minutes*) **Redéfinition de méthodes**
Redéfinissez la méthode `__str__()` pour produire une représentation textuelle de vos objets `Fraction`.

Question 5: (🕓 *5 minutes*) **Accesseurs et mutateurs**
Créez des `getters` et des `setters` pour les attributs de votre classe `Fraction`.

Question 6: (🕓 *10 minutes*) **Simplification de fractions**
Définissez une méthode `simplification()` qui rend la fraction irréductible en modifiant ses attributs. Cette méthode ne renvoie aucune valeur. Appelez-la après chaque opération afin de conserver des fractions simplifiées.

Question 7: (🕓 *15 minutes*) **Redéfinition de méthodes**
Redéfinissez la méthode `__eq__(self, other)` pour comparer deux fractions. La méthode doit renvoyer `True` si elles représentent le même nombre et `False` sinon. Vérifiez le type de `other` avec `isinstance(other, Fraction)` et affichez un message d'erreur sinon.

Question 8: (🕓 *15 minutes*) **Addition et multiplication**
Redéfinissez `__add__` et `__mul__` pour additionner et multiplier les objets `Fraction`, en renvoyant une `Fraction` *simplifiée*. Gérez les arguments qui ne sont ni des `Fraction` ni des `int`.

2 Polymorphisme en Java

Les exercices de cette section s'inspirent des exercices de la section 1 des exercices avancés de la semaine 6. N'hésitez pas à réutiliser les solutions disponibles sur Moodle.
Dans cette partie, vous devez créer deux nouvelles classes filles de la classe-mère `Fighter`. La première classe représentera un `Soigneur`, qui, lorsqu'il "attaquera" un `Fighter`, le soignera au lieu de le blesser. La deuxième classe, nommée `Attaquant`, représentera un `Fighter` spécialisé dans l'attaque et aura la

capacité d'attaquer un certain nombre de fois (ce nombre sera défini lors de l'instanciation).

Avant d'effectuer la série de questions suivantes, téléchargez le fichier `Fighter.java` disponible sur Moodle.

Question 9: (🕓 *5 minutes*) **Classe-fille `Soigneur`**
Créez une nouvelle classe-fille `Soigneur` qui hérite de la classe `Fighter`. Cette classe-fille aura un nouvel attribut privé nommé `resurrection` qui vaudra 1 lors de l'instanciation de la classe.

Créez le constructeur de cette classe ainsi que le `getter` et le `setter` permettant d'interagir avec le nouvel attribut (`resurrection`).

Question 10: (🕓 *15 minutes*) **Méthode `resurrection(Fighter other)`**
Créez une méthode `resurrection(Fighter other)` qui permettra de faire revenir un `Fighter` à la vie, mais le `Soigneur` ne pourra le faire qu'une seule fois.

Commencez par contrôler que l'instance depuis laquelle la méthode est appelée est toujours en vie. Si ce n'est pas le cas, indiquez : `nom_instance` est mort et ne peut plus rien faire.

Contrôlez ensuite que l'instance `other` est vraiment morte. Si ce n'est pas le cas, indiquez-le en affichant le texte suivant : "`nom_other` est toujours en vie."

Pour finir, contrôlez que l'attribut `resurrection` de l'instance depuis laquelle la méthode est appelée est égal à 1. Si ce n'est pas le cas, indiquez : `nom_instance` ne peut plus ressusciter personne.

Si tous ces éléments sont réunis, faites revenir le `Fighter other` à la vie en lui remettant 10 points de vie et en l'ajoutant à la liste `instances` de la classe `Fighter`. Pensez aussi à :

- mettre l'attribut `resurrection` de l'instance appelée à 0 afin de l'empêcher de réutiliser ce pouvoir,
- appeler la méthode `checkHealth()` et indiquer : "`nom_other` est revenu à la vie !"

Question 11: (🕓 *10 minutes*) **Méthode `attack`**

Réécrivez la méthode `attack` de la classe-fille `Soigneur` afin d'ajouter des points de vie à `other` au lieu de lui en retirer.

Le seul argument nécessaire pour cette méthode sera une autre instance de `Fighter` qu'on pourrait nommer `other`.

Commencez par contrôler que le `Soigneur` depuis lequel la méthode est appelée est encore en vie. Si ce n'est pas le cas, affichez : "`nom_instance` est mort et ne peut plus rien faire."
Contrôlez ensuite que le `Fighter other` est toujours en vie. Si ce n'est pas le cas, indiquez-le : "`nom_other` est déjà mort, ressuscitez-le afin de pouvoir le soigner." Contrôlez également qu'il a moins de 10 points de vie. Si ce n'est pas le cas, indiquez-le via le texte suivant : "`nom_other` a déjà le maximum de points de vie."

Si toutes ces conditions sont réunies, ajoutez la valeur de l'attaque de l'instance qui appelle la méthode aux points de vie de `other`, puis appelez la méthode de classe `checkHealth()`.

Question 12: (🕓 *5 minutes*) **Classe-fille `Attaquant`**

Commencez par déclarer une nouvelle classe-fille `Attaquant` héritant de la classe `Fighter`. Cette classe-fille prendra un nouvel attribut privé nommé `multiplicateur` de type `int`, qui sera passé comme argument du `constructeur` de la classe-fille.

Déclarez le `constructeur` de cette classe ainsi que le `getter` et le `setter` permettant d'interagir avec ce nouvel attribut `multiplicateur`.

Question 13: (*10 minutes*) **Méthode `attack`**

Réécrivez la méthode `attack` de la classe-fille `Attaquant` afin d'effectuer plusieurs attaques sur `other` en fonction de l'attribut `multiplicateur`.

Assurez-vous de toujours indiquer le type de l'attaque lorsque vous faites appel à la méthode `attack()`.

Pour vérifier votre programme, exécutez la méthode `main` suivante :

```
public class FighterMain {
    public static void main(String[] args) {
        Fighter P1 = new Fighter("P1", 10, 2, 2);
        Attaquant P2 = new Attaquant("P2", 10, 2, 2, 2);
        Soigneur P3 = new Soigneur("P3", 10, 4, 2);
        P1.attack("pied", P2);
        P1.attack("poing", P2);
        P1.attack("tete", P2);
        P1.attack("tete", P2);
        P3.resurrection(P2);
        P1.attack("pied", P2);
        P1.attack("poing", P2);
        P1.attack("tete", P2);
        P3.attack(P2);
        P2.attack("tete", P1);
    }
}
```

Vous devriez obtenir :

```
P1 a encore 10 points de vie
P2 a encore 8 points de vie
P3 a encore 10 points de vie
------------------------------
P1 a encore 10 points de vie
P2 a encore 6 points de vie
P3 a encore 10 points de vie
------------------------------
P1 a encore 10 points de vie
P2 a encore 2 points de vie
P3 a encore 10 points de vie
------------------------------
P2 est mort
P1 a encore 10 points de vie
P3 a encore 10 points de vie
------------------------------
P2 vient de revenir à la vie
P1 a encore 10 points de vie
P3 a encore 10 points de vie
P2 a encore 10 points de vie
------------------------------
P1 a encore 10 points de vie
P3 a encore 10 points de vie
P2 a encore 8 points de vie
------------------------------
P1 a encore 10 points de vie
P3 a encore 10 points de vie
P2 a encore 6 points de vie
------------------------------
P1 a encore 10 points de vie
P3 a encore 10 points de vie
P2 a encore 2 points de vie
------------------------------
P1 a encore 10 points de vie
```

```
P3 a encore 10 points de vie
P2 a encore 6 points de vie
------------------------------
Attaque n 1
P1 a encore 6 points de vie
P3 a encore 10 points de vie
P2 a encore 6 points de vie
------------------------------
Attaque n 2
P1 a encore 2 points de vie
P3 a encore 10 points de vie
P2 a encore 6 points de vie
------------------------------
```

3 Notions d'héritage en Python

Question 14: (*15 minutes*) **Classe Point (Suite)**
La semaine précédente, vous avez défini une classe `Point` représentant un point en deux dimensions, avec des coordonnées `x` et `y`, ainsi que des opérations basiques sur des points 2D. Pour cet exercice, vous allez implémenter une classe de points à trois dimensions qui héritera de la classe `Point` créée précédemment.

Avant de commencer, rendez les attributs de la classe `Point` protégés plutôt que privés afin qu'ils soient accessibles sans getters ou setters.

Écrivez une classe qui hérite de `Point`. Nommez-la `Point3D`. Après avoir rajouté la troisième dimension comme attribut, effectuez les opérations ci-dessous :

- Définissez une méthode qui renvoie les trois coordonnées du point dans une liste.
- Recalculez la distance euclidienne et le milieu pour le point 3D.
- Pour aller plus loin Si vous voulez vous familiariser encore plus avec les méthodes de classe en Python, implémentez deux autres algorithmes de calcul de distance : les distances de Manhattan et Minkowski.

Vous pouvez compléter le code suivant (qui se trouve sur Moodle, dans le dossier `Code`).

```python
import math
import Point

class Point3D(Point):
    def __init__(self, x, y):
        super().__init__(x, y)
        # Rajoutez un nouvel attribut z propre à la classe Point3D


    # Définir le getter et le setter sur le nouvel attribut z

    # Complétez les méthodes suivantes
    def vector_representation(self): # représentée sous forme de liste
        ...

    def distance(self, p2): # i.e norme euclidéene
        ...

    def distance_manhattan(self, p2):
        ...

    def distance_minkowski(self, p2, order=3):
        ...

    def milieu(self, p2):
        ...
```